基于 CLAUDE.md 提炼

AI Agent 原则文件讲解

让 AI 更像靠谱工程师，而不是写得很快、但经常误解需求、过度设计、改动失控、无法验证的助手。

内容概览

第一部分：问题

AI 写代码的常见失误

为什么需要原则约束

四条原则的总框架

第二部分：原则

Think Before Coding

Simplicity First

Surgical Changes

Goal-Driven Execution

第三部分：落地

如何写约束型 Prompt

怎样减少返工和回归

如何建立对 AI 的信任

第四部分：例子

错误做法与正确做法对比

更贴近程序员日常场景

第五部分：带走

可直接复用的 Prompt 模板

分享时最值得强调的结论

第一部分：这份原则文件在解决什么问题

先说明痛点，再讲原则，听众更容易接受

从“AI 为什么经常翻车”开始讲，听众会更快进入状态

AI 写代码最常见的四类失误

CLAUDE.md 的价值，不在于“让 AI 更会写”，而在于“让 AI 少犯工程性错误”。

1. 没想清楚就开写

误解需求

偷偷做假设

把不确定当确定

2. 过度设计

为未来扩展先造框架

为小问题造系统

代码量远大于问题本身

3. 改动失控

顺手重构无关代码

diff 很大但价值不集中

回归风险显著上升

4. 做完但没验证

“应该可以”

没有成功标准

没有测试闭环

四条原则的总框架

不是教 AI “多做一点”，而是要求它按更像工程师的方式工作。

Think Before Coding

先理解问题，明确假设，不要带着误解直接进入实现。

Simplicity First

优先选择最小实现，不为未来虚构需求预支复杂度。

Surgical Changes

控制改动面，只改必要位置，只清理自己造成的影响。

Goal-Driven Execution

把模糊任务转成可验证目标，完成必须有证据闭环。

第二部分：四条原则分别在约束什么

每条原则都对应一种高频失误

理解它们的顺序关系，比背定义更重要

原则 1：Think Before Coding

不要想当然，不要掩盖困惑，把假设和分歧点讲出来。

它约束的是什么

AI 在开始编码之前的思考质量。

你希望它先交代：它理解的需求、关键假设、可选路径，以及推荐哪条路径。

它真正带来的价值

把“错误地快速行动”变成“明确地慢 10 秒”。

程序员先确认 AI 对问题空间的理解，再让它进入解法空间。

错误方式

User：给用户列表加筛选
AI：好的，我加上按姓名、状态、时间范围、多字段排序、导出 CSV

正确方式

我理解的目标是：
1. 增加筛选能力

原则 2：Simplicity First

最少的代码，解决当前问题。不要预支未来复杂度。

文件里的硬约束

不加没被要求的特性

不为单次使用抽象

不为了“灵活”先造配置系统

不为不可能场景堆复杂错误处理

判断标准

如果 50 行能解决，就别写成 200 行。

简单不是能力弱，而是控制力强；更小的实现更容易 review、测试和维护。

过度设计

```
interface ValidatorStrategy {}
class EmailValidatorFactory {}
class ValidationRegistry {}
class ConfigurableRuleEngine {}
```

合理实现

```
function isValidEmail(email: string) {
  return /.../.test(email)
}
```

在第二、第三类规则出现之前，先别造系统。

原则 3 与原则 4

一个控制改动面，一个保证结果可验证，二者一起决定了 AI 的工程可靠性。

Surgical Changes

只改必须改的地方，只清理自己造成的影响。

防的是：顺手优化、顺手重构、顺手统一格式、顺手删历史代码。

例子：修复登录按钮无响应时，只修 `onClick` 绑定和必要测试，而不是顺手重写整个表单组件。

Goal-Driven Execution

任务不是“做点什么”，而是“达到一个可验证的结果”。

“修 bug” 要先复现，再修复，再验证。

没有验证标准，就没有真正完成。

```
1. 复现问题 -> verify: 测试失败或步骤可稳定复现
2. 修改实现 -> verify: 测试通过
```

第三部分：怎么给团队落地

从“理解原则”走到“能真正拿来用”

重点不是讲概念，而是给团队一套能复制的用法

把四条原则翻译成团队可落地的方法

阶段	AI 应该做什么	产出
理解	复述需求、列假设、说风险	问题定义
设计	比较方案、选最简单路径	最小方案
实施	小范围修改、遵循现有风格	可 review 的 diff
验证	测试、复现、检查回归	完成证据

三个实操建议

先让 AI 复述任务

明确要求最小改动

要求给出验证方式

最值得强调的结论

没有原则的 AI 错得又快又大。

有原则的 AI 看起来慢一点，但交付更稳，长期更快。

可直接复用的 Prompt 模板

好的 AI Agent 原则，不是让模型更聪明

而是让它在工程协作里更可靠

先想清楚 / 保持简单 / 精准改动 / 验证闭环